Итоговый проект по дисциплине
«Автоматизация процессов разработки и тестирования моделей машинного обучения»

PD-модель: предсказание дефолта клиентов кредитных карт

Демонстрация работы проекта

Студент: Антон Кислинский

Репозиторий: https://github.com/orenkant/MIFI_Track2_itog

Датасет: Default of Credit Card Clients (UCI)

Ноябрь 2026

Что показано в документе

Скриншоты сняты в итоговой папке проекта: после `git clone` и `dvc pull` пайплайн был перезапущен полностью (`dvc repro -f`), команды и их вывод настоящие. Разделы идут в порядке критериев оценки из задания.

Критерий	Раздел	Что показано
Организация кода и Git	1	структура репозитория, история коммитов
Подготовка и валидация данных	2	EDA, новые признаки, проверки Great Expectations
Построение и настройка модели	3	обучение пяти моделей, метрики, ROC-кривые
MLflow Tracking	4	запуски, сравнение, артефакты, реестр моделей
DVC	5	граф стадий, dvc repro, реакция на изменения, хранилище
Тестирование и CI	6	pytest, линтеры, прогон GitHub Actions
Docker и FastAPI	7	сборка образа, контейнер, запросы к API, Swagger
Мониторинг (дрифт)	8	PSI на обычном потоке и на потоке со сдвигом

Итоговые результаты (тестовая выборка, 5 993 клиента):

Модель	CV ROC-AUC	ROC-AUC	Gini	Precision	Recall	F1
hist_gradient_boosting (выбрана)	**0.788**	0.777	0.555	0.503	0.575	0.537
gradient_boosting	0.788	0.777	0.554	0.519	0.561	0.539
random_forest	0.787	0.775	0.551	0.529	0.534	0.531
logreg_fe	0.763	0.757	0.514	0.480	0.574	0.523
logreg_baseline	0.726	0.719	0.438	0.555	0.447	0.495

1. Организация кода и Git

Структура взята из рекомендованной в задании: `src/data`, `src/features`, `src/models`, `src/api`, `tests`, `notebooks`, `data`, `models`. Дополнительно `src/monitoring` для расчёта дрифта.

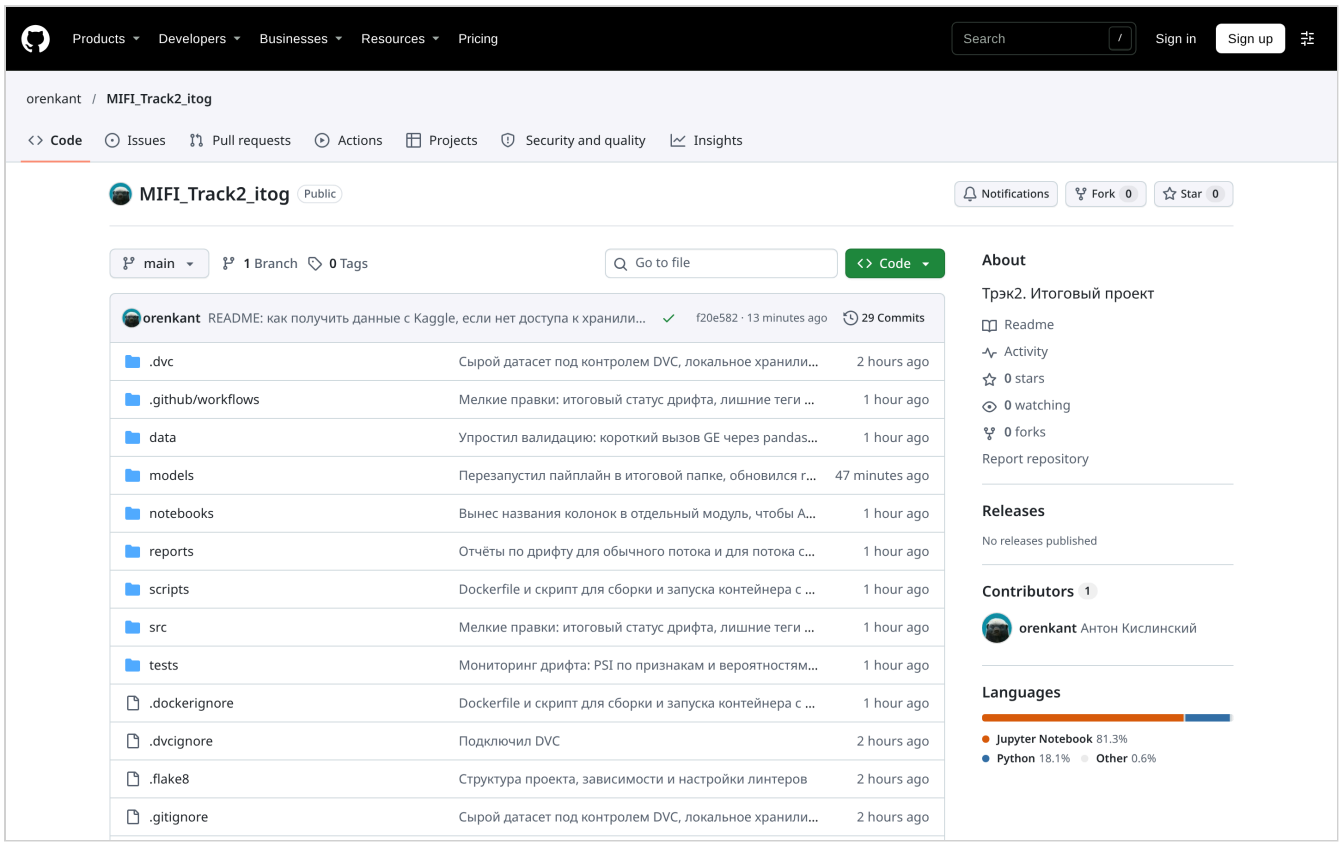


Рис. 1.1. Репозиторий на GitHub

> Данные и модель в git не лежат: вместо них файлы-указатели DVC (`data/raw/UCI_Credit_Card.csv.dvc`, `dvc.lock`).

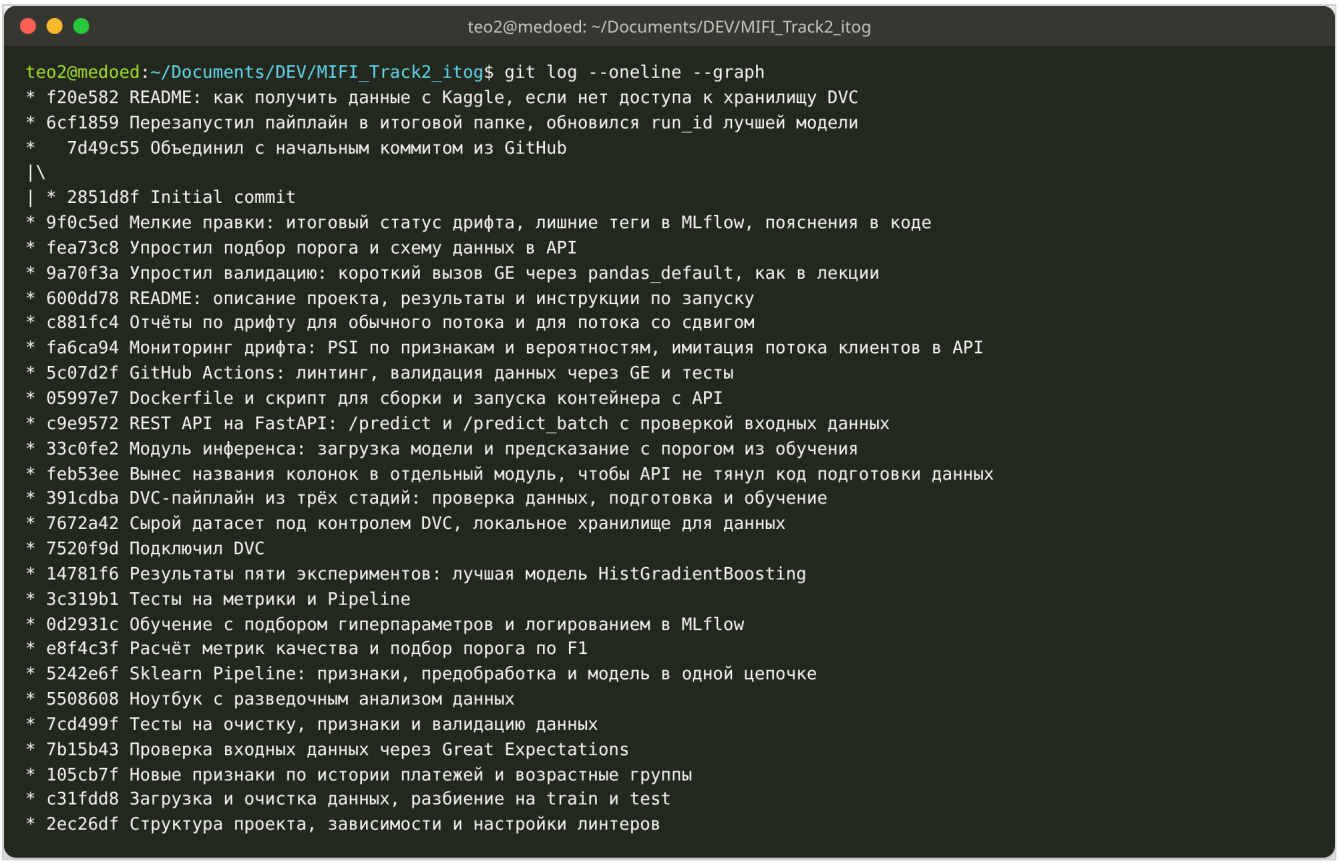


Рис. 1.2. История коммитов

> Коммиты делались по этапам: данные → модель → MLflow → DVC → API и Docker → CI и мониторинг, в конце упрощения после ревью. Ветка справа – начальный коммит, который GitHub создал вместе с репозиторием, она слита с основной.

2. Подготовка и валидация данных

30 000 клиентов, 23 признака, пропусков нет, дефолтов около 22%. Очистка в `src/data/make_dataset.py`, новые признаки в `src/features/build_features.py`, проверки в `src/data/validation.py`.

In [5]:
```
target_share = df[RAW_TARGET].value_counts(normalize=True).rename("доля")
print(target_share.round(3))

target_share.plot(kind="bar", title="Распределение таргета", rot=0)
plt.show()
```

```
default.payment.next.month
0    0.779
1    0.221
Name: доля, dtype: float64
```

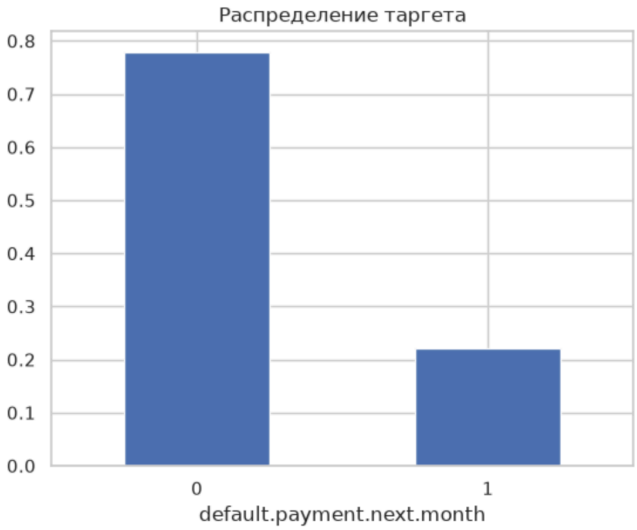


Рис. 2.1. EDA: баланс классов

> Классы несбалансированы, поэтому разбиение со стратификацией, а главная метрика – ROC-AUC (accuracy константной модели была бы 0.78).

In [6]:
```
for col in ["SEX", "EDUCATION", "MARRIAGE"]:
    stat = df.groupby(col)[RAW_TARGET].agg(["count", "mean"]).rename(
        columns={"count": "клиентов", "mean": "доля дефолтов"}
    )
    print(stat.round(3), end="\n\n")
```

```
         клиентов  доля дефолтов
SEX
1           11888          0.242
2           18112          0.208

           клиентов  доля дефолтов
EDUCATION
0                14          0.000
1             10585          0.192
2             14030          0.237
3              4917          0.252
4               123          0.057
5               280          0.064
6                51          0.157

          клиентов  доля дефолтов
MARRIAGE
0               54          0.093
1            13659          0.235
2            15964          0.209
3              323          0.260
```

Рис. 2.2. EDA: категориальные признаки

> В `EDUCATION` есть коды 0, 5, 6, а в `MARRIAGE` код 0, которых нет в описании датасета. При очистке объединил их с категорией «прочее».

```
In [7]: fig, ax = plt.subplots(figsize=(8, 4))
df.groupby("PAY_0")[RAW_TARGET].mean().plot(kind="bar", ax=ax, rot=0)
ax.set_title("Доля дефолтов по статусу платежа за сентябрь (PAY_0)")
ax.set_ylabel("доля дефолтов")
plt.show()
```



Рис. 2.3. EDA: статус платежа за последний месяц

> Самый сильный исходный признак. При просрочке от двух месяцев доля дефолтов 0.6–0.7, у тех, кто платит вовремя, около 0.13.

```
In [11]: fe = add_features(clean_data(df))
fe[NEW_NUMERIC_FEATURES + ["target"]].corr(method="spearman")["target"].drop("target").round(3)

Out[11]: MONTHS_DELAYED    0.388
MAX_DELAY         0.373
UTILIZATION       0.073
BILL_MEAN        -0.027
PAY_AMT_MEAN     -0.175
PAY_RATIO        -0.113
Name: target, dtype: float64
```

Рис. 2.4. Новые признаки: корреляция Спирмена с таргетом

> `MONTHS_DELAYED` (число месяцев с просрочкой) и `MAX_DELAY` связаны с дефолтом сильнее любого исходного признака (у `PAY_0` 0.29). Все признаки считаются построчно, поэтому утечки из теста нет.

```
In [12]: fig, axes = plt.subplots(1, 2, figsize=(12, 4))
fe.groupby("MONTHS_DELAYED")["target"].mean().plot(kind="bar", ax=axes[0], rot=0)
axes[0].set_title("Доля дефолтов по числу месяцев с просрочкой")
order = ["<25", "25-34", "35-44", "45-54", "55-64", "65+"]
fe.groupby("AGE_BIN")["target"].mean().reindex(order).plot(kind="bar", ax=axes[1], rot=0)
axes[1].set_title("Доля дефолтов по возрастным группам")
plt.show()
```

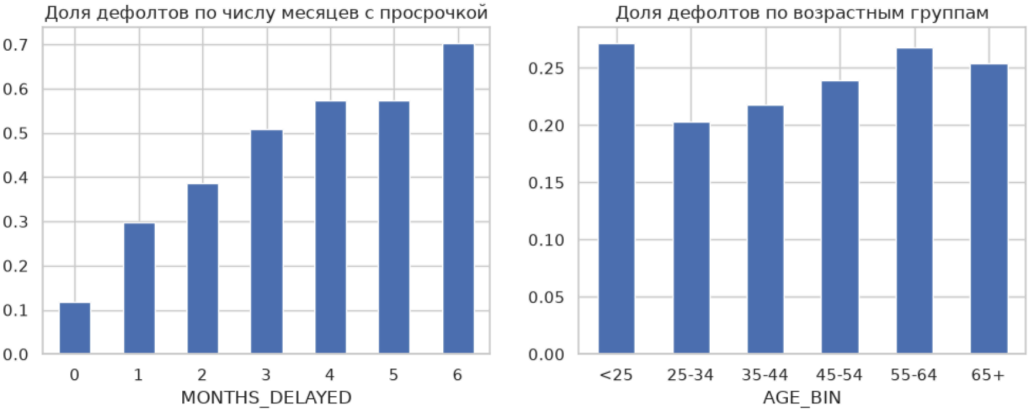


Рис. 2.5. Доля дефолтов по новым признакам

Справа видно, почему возраст разбит на группы: зависимость нелинейная, риск выше у самых молодых и у 55+.

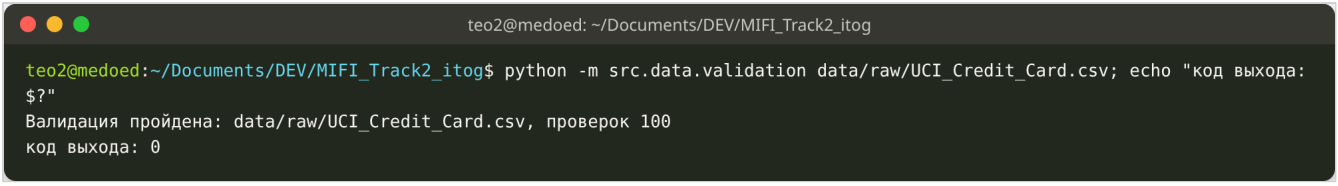


Рис. 2.6. Great Expectations: проверка исходного датасета

Набор из 100 правил: колонки на месте и без пропусков, типы, допустимые коды категорий, диапазоны значений, уникальность `ID`, таргет только 0/1.

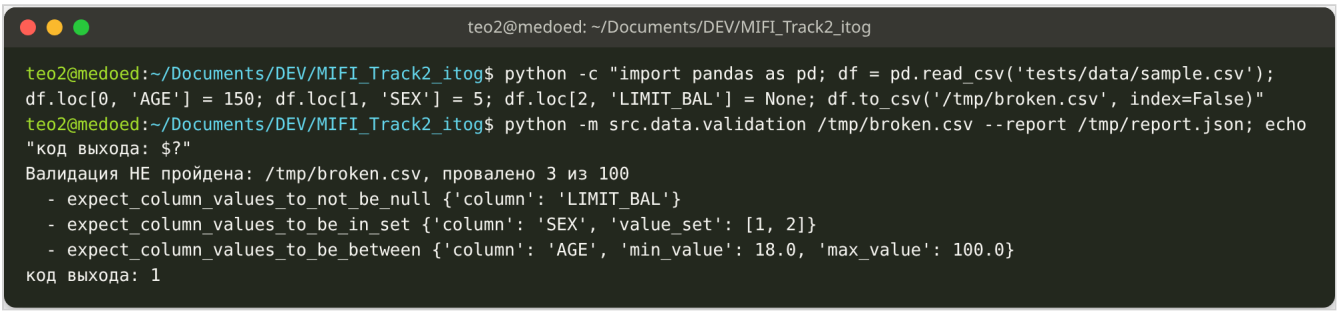


Рис. 2.7. Great Expectations: испорченные данные

В выборку подставлены `AGE=150`, `SEX=5` и пропуск в `LIMIT_BAL`. Все три ошибки найдены, скрипт вернул код 1 – на этом останавливаются DVC-пайплайн и CI.

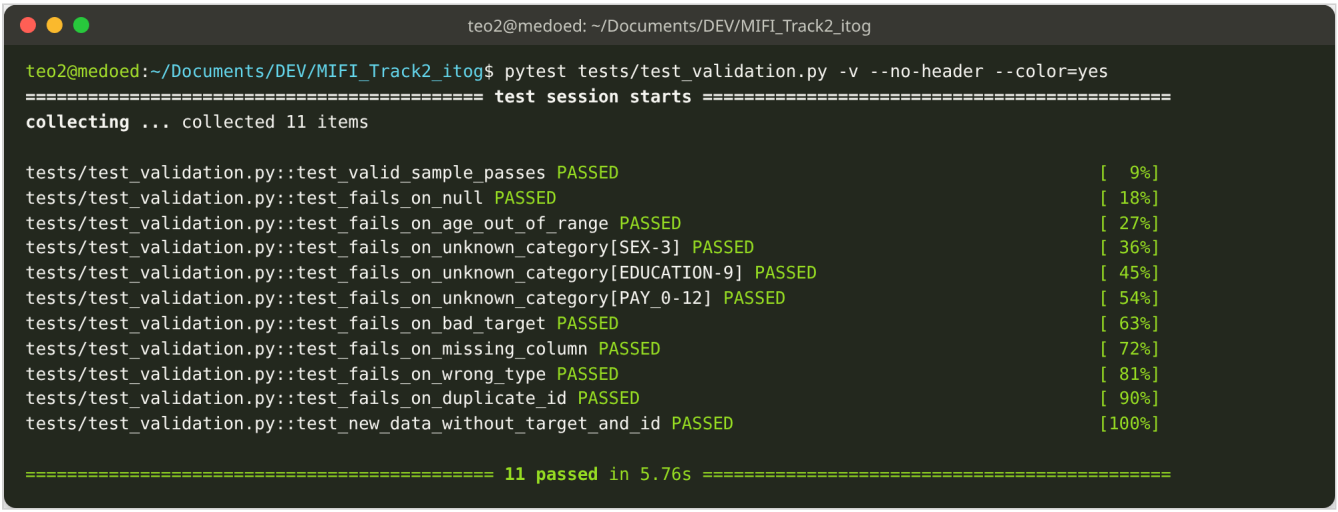


Рис. 2.8. Тесты валидации

Каждый тест портит данные по-своему и проверяет, что валидация не прошла. Последний тест – режим для новых данных без таргета и `ID`.

3. Построение и настройка модели

Pipeline: добавление признаков (`FunctionTransformer`) → `ColumnTransformer` (`SimpleImputer` + `StandardScaler` для числовых, `SimpleImputer` + `OneHotEncoder` для категориальных) → модель. Гиперпараметры подбираются через `GridSearchCV` / `RandomizedSearchCV` на 5 стратифицированных фолдах, сетки в `params.yaml`.

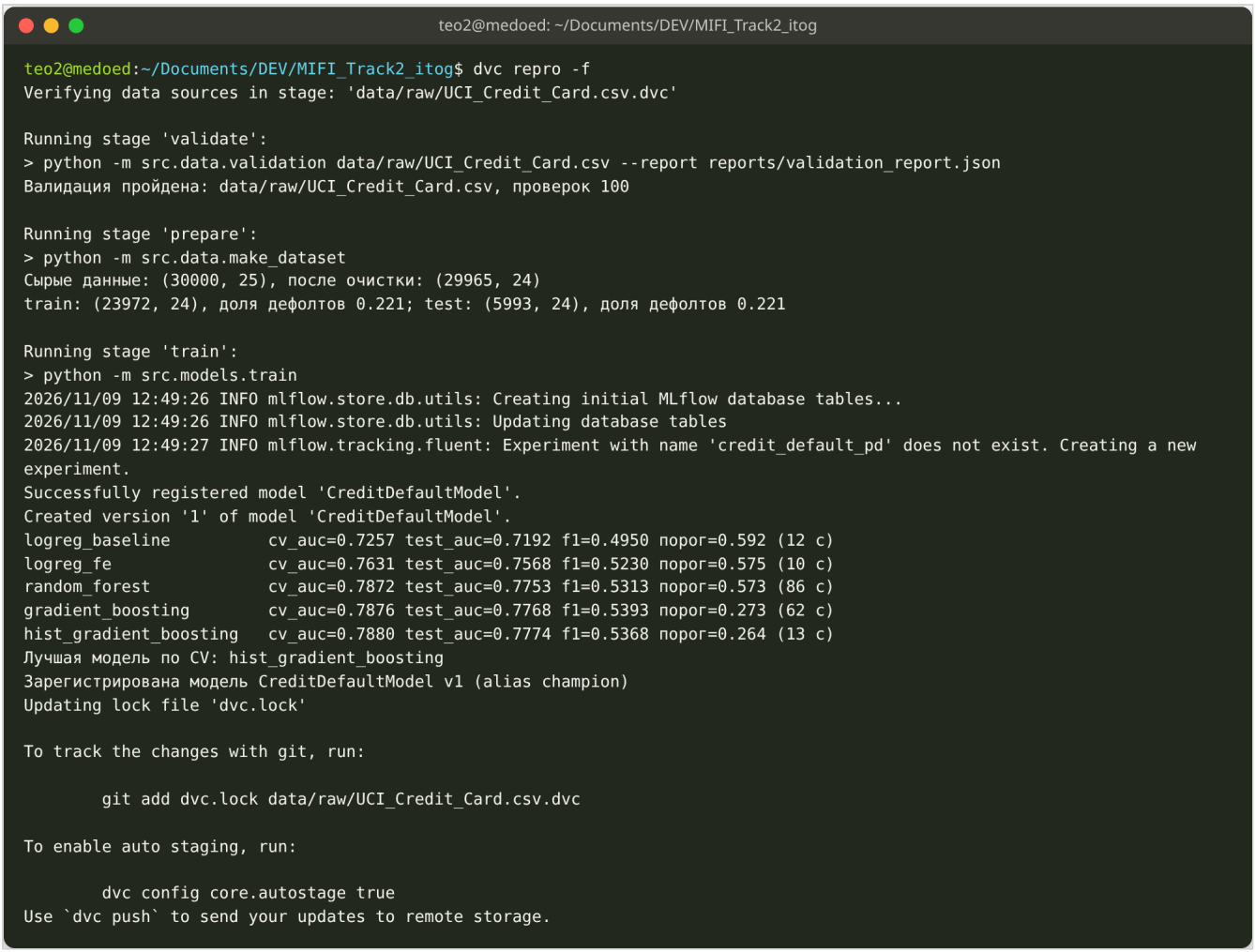


Рис. 3.1. Полный прогон пайплайна: проверка, подготовка, обучение пяти моделей

> Лучшая модель выбирается по ROC-AUC на кросс-валидации, тест используется только для итоговой оценки. Порог классификации подобран по F1 на out-of-fold предсказаниях train. Метрики совпадают с предыдущими запусками – обучение воспроизводимо.

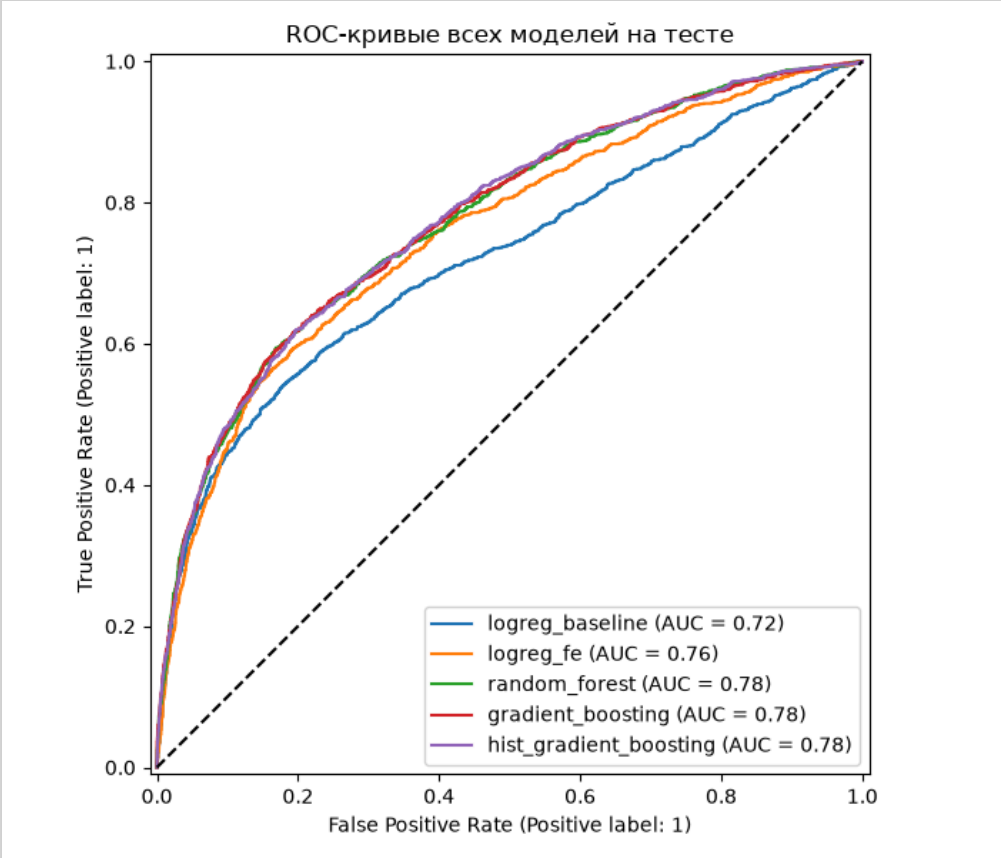


Рис. 3.2. ROC-кривые всех моделей на тестовой выборке

> Новые признаки дают логистической регрессии +0.04 ROC-AUC. Три ансамбля почти совпадают, выбран HistGradientBoosting: лучший CV и обучение в 5–7 раз быстрее остальных.

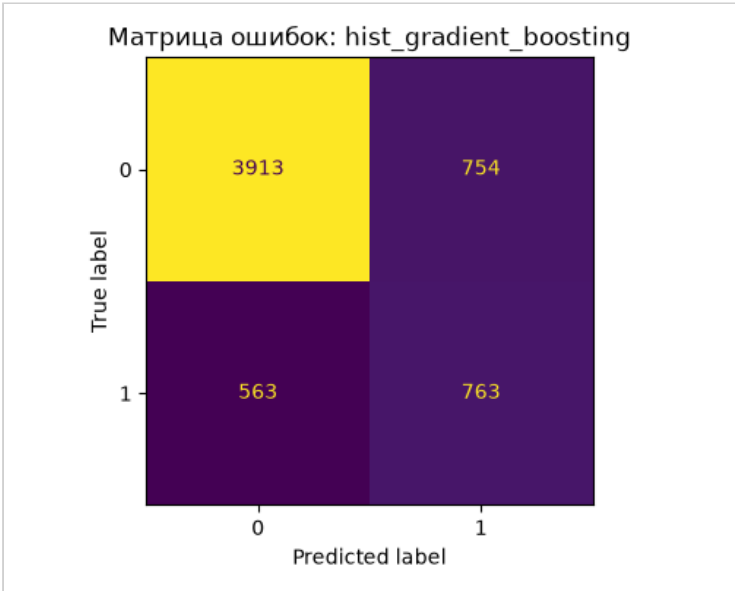


Рис. 3.3. Матрица ошибок лучшей модели (порог 0.264)

> Модель находит 763 из 1 326 дефолтов (recall 0.58), из помеченных как риск половина действительно дефолтит (precision 0.50).

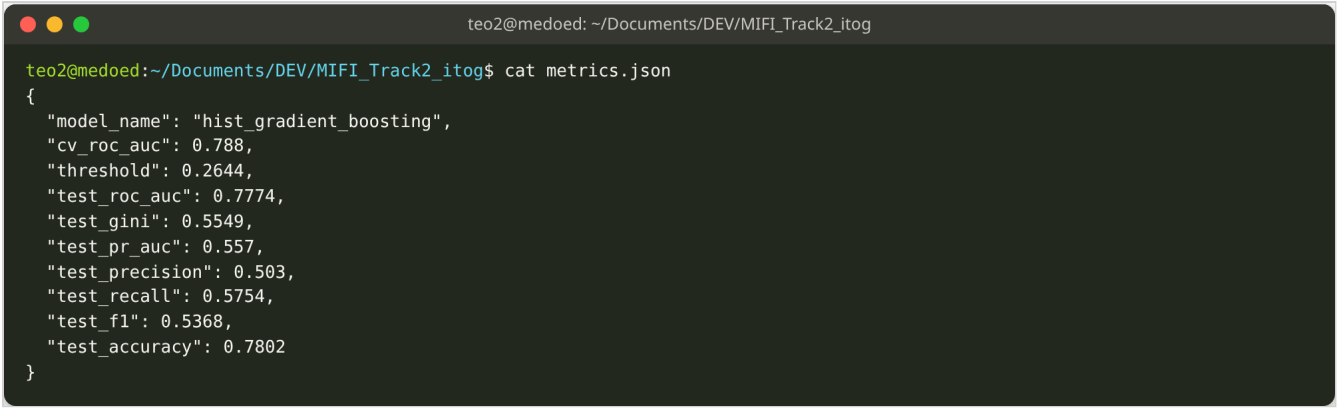


Рис. 3.4. Итоговые метрики (metrics.json, его же показывает dvc metrics show)

> Gini = 2·AUC − 1 – метрика, которую принято смотреть в кредитном скоринге.

4. MLflow Tracking

Каждый эксперимент – отдельный run: параметры, метрики CV и теста, артефакты (ROC-кривая, матрица ошибок, результаты подбора, отчёт) и сама модель через `mlflow.sklearn.log_model`. Хранилище – `sqlite:///mlflow.db`, интерфейс – `mlflow ui` (MLflow 3.18, режим Model training).

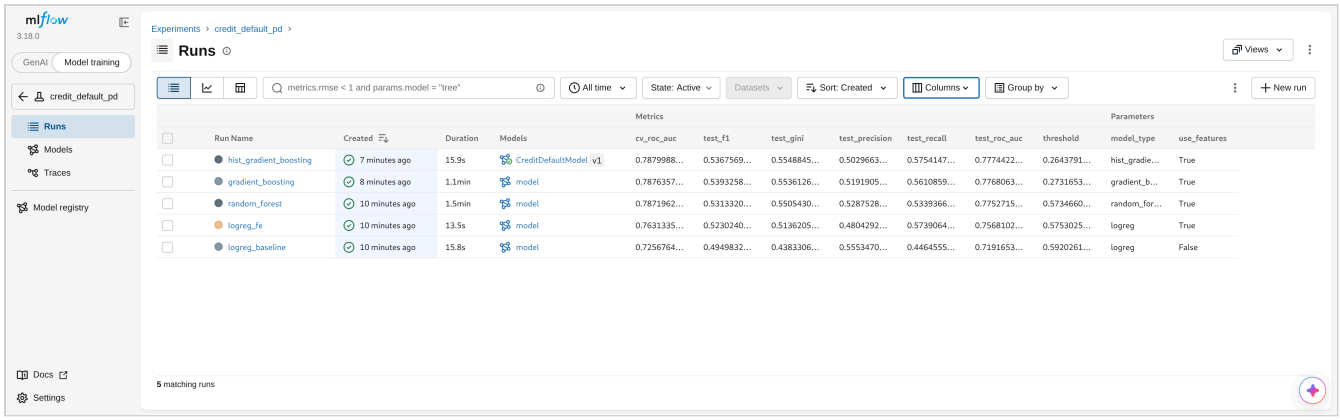


Рис. 4.1. Пять запусков эксперимента credit_default_pd

> Один запуск на эксперимент из `params.yaml`. В колонках метрики CV и теста, тип модели и флаг `use_features` (у baseline признаки не добавлялись). Лучшая модель сразу помечена как `CreditDefaultModel v1`.

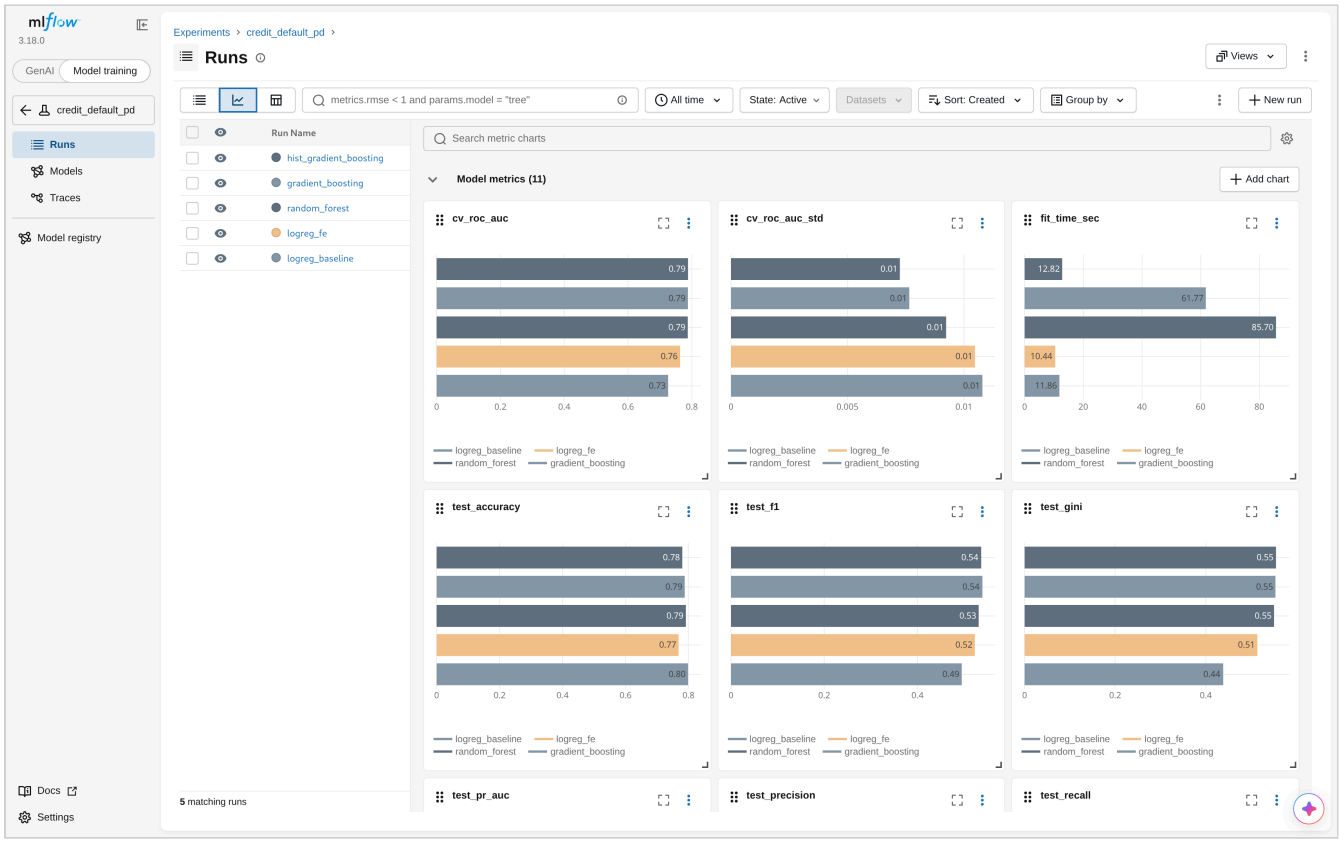


Рис. 4.2. Сравнение запусков на графиках

По `cv_roc_auc` три ансамбля почти равны (0.79), логистическая регрессия с новыми признаками – 0.76, без них – 0.73. По `fit_time_sec` видно, что HistGradientBoosting учится быстрее остальных ансамблей (13 с против 62 и 86 с), поэтому при равном качестве выбран он.

Run details

Run ID:	83d907000f174c6db6a33f…	3aab4c6b3945417ab637b…	67c61935759848498d964…	abd8484789a847b0bb5e0…	e2f6f1f537344788a400ce2…
Run Name:	hist_gradient_boosting	gradient_boosting	random_forest	logreg_fe	logreg_baseline
Start Time:	11/09/2026, 12:52:33 PM	11/09/2026, 12:51:27 PM	11/09/2026, 12:49:57 PM	11/09/2026, 12:49:43 PM	11/09/2026, 12:49:28 PM
End Time:	11/09/2026, 12:52:48 PM	11/09/2026, 12:52:33 PM	11/09/2026, 12:51:27 PM	11/09/2026, 12:49:57 PM	11/09/2026, 12:49:43 PM
Duration:	15.9s	1.1min	1.5min	13.5s	15.8s

Parameters

Show diff only

C				0.1	1.0
class_weight			balanced_subsample	balanced	balanced
cv_folds	5	5	5	5	5
l1_ratio				1	1
l2_regularization	1.0				
learning_rate	0.03	0.05			
max_depth		4	10		
max_features			sqrt		
max_iter	400				
max_leaf_nodes	31				
min_samples_leaf	100		20		

Рис. 4.3. Страница сравнения: подобранные гиперпараметры

У логистической регрессии лучшие C=0.1 (с новыми признаками) и C=1.0 (без них), в обоих случаях L1-регуляризация (`l1_ratio=1`) и `class_weight=balanced` . У бустингов подобрался небольшой шаг обучения (0.03 и 0.05).

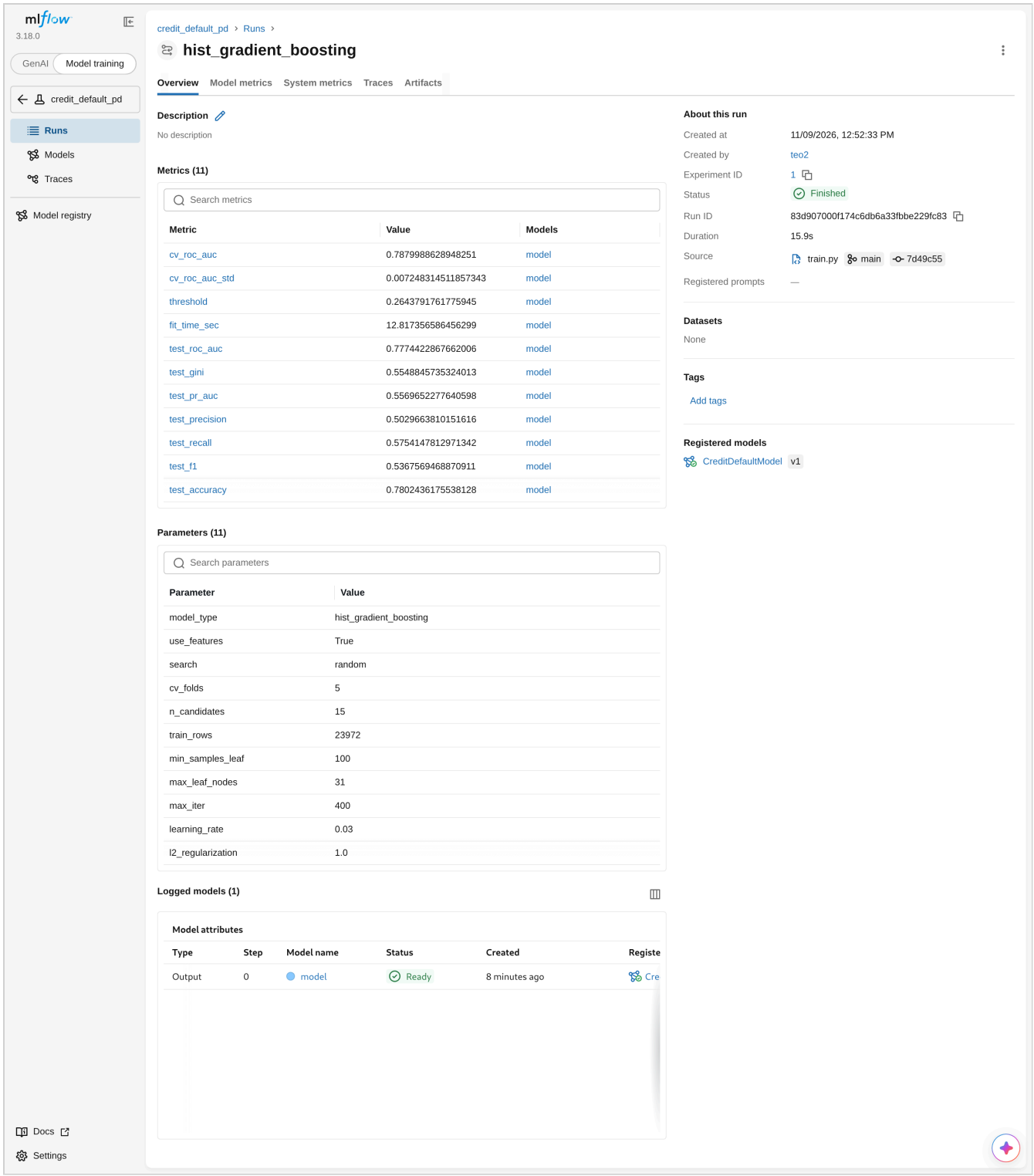


Рис. 4.4. Карточка лучшего запуска

> Все 11 метрик, ссылка на зарегистрированную модель и коммит git, из которого шло обучение (`main` , `7d49c55`) – по нему всегда можно найти код, давший эту модель.

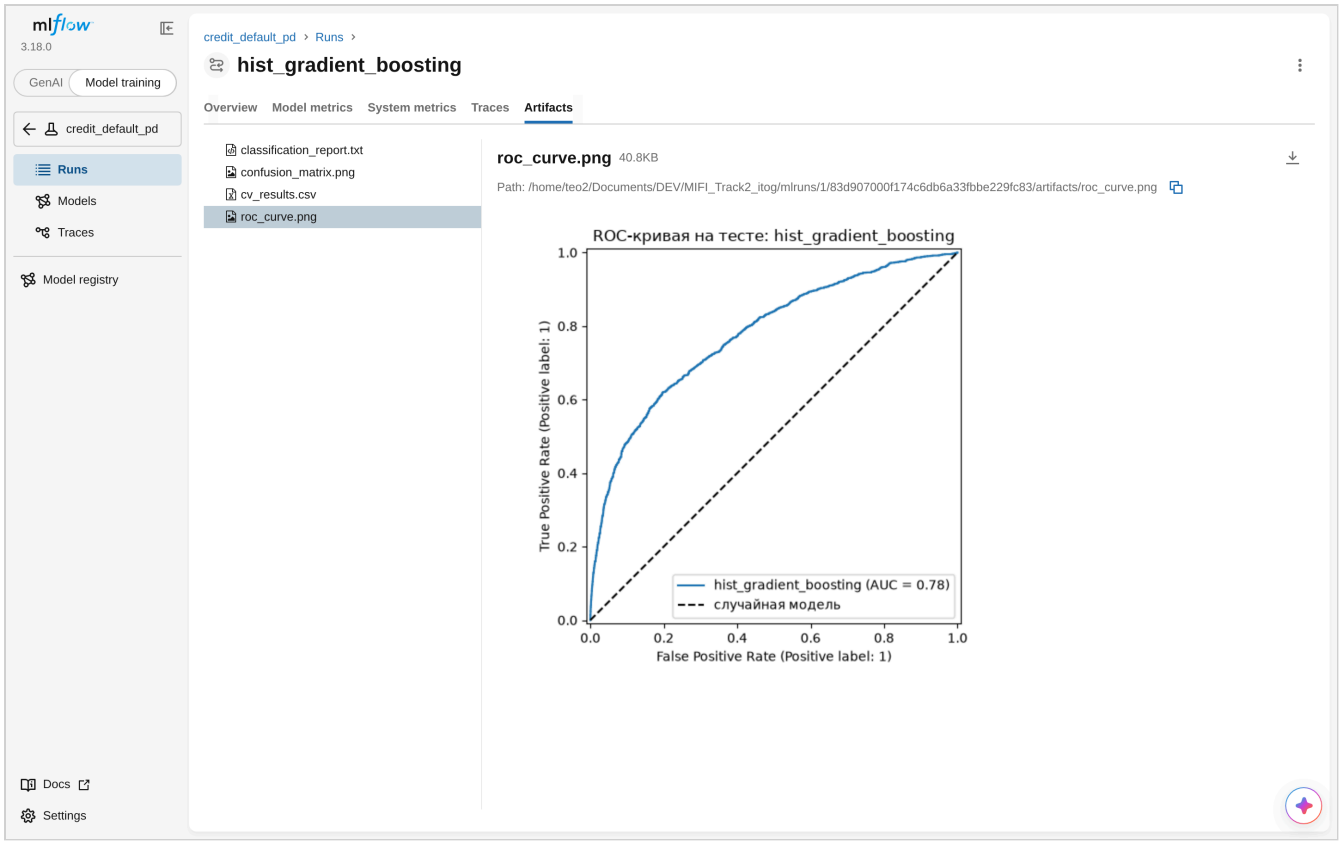


Рис. 4.5. Артефакты запуска

> Отчёт по классам, матрица ошибок, результаты подбора гиперпараметров и ROC-кривая. Графика важности признаков у этой модели нет: у HistGradientBoosting нет встроенной оценки важности, у остальных моделей он сохраняется.

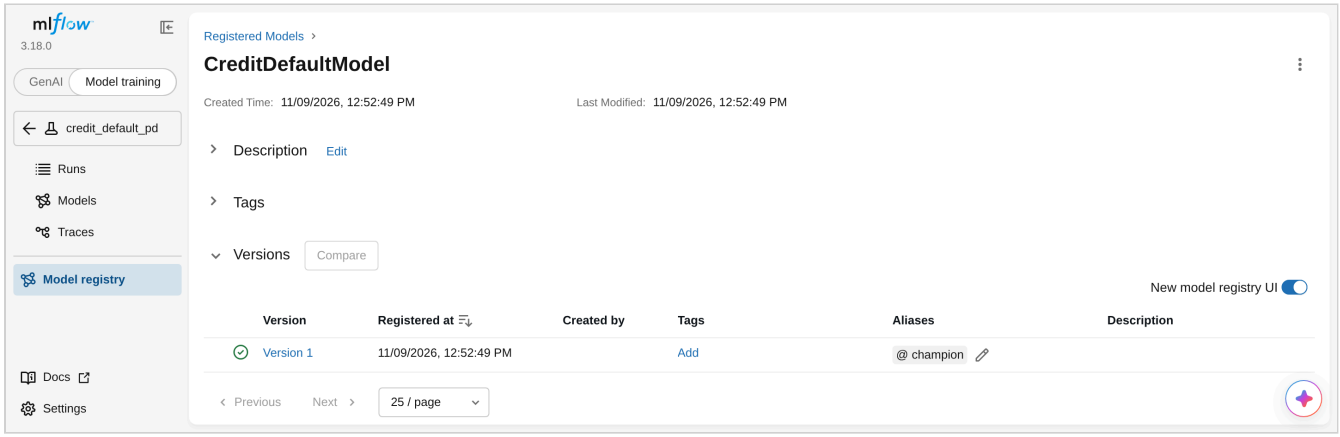


Рис. 4.6. Реестр моделей

> Лучшая по CV модель регистрируется автоматически и получает алиас `champion`. В новых версиях MLflow алиасы заменили стадии Staging/Production из лекции.

5. Контроль версий данных и моделей (DVC)

Исходный датасет добавлен через `dvc add`, пайплайн описан в `dvc.yaml`: `validate → prepare → train`. Модель `models/model.pkl` и подготовленные данные – выходы стадий под DVC.

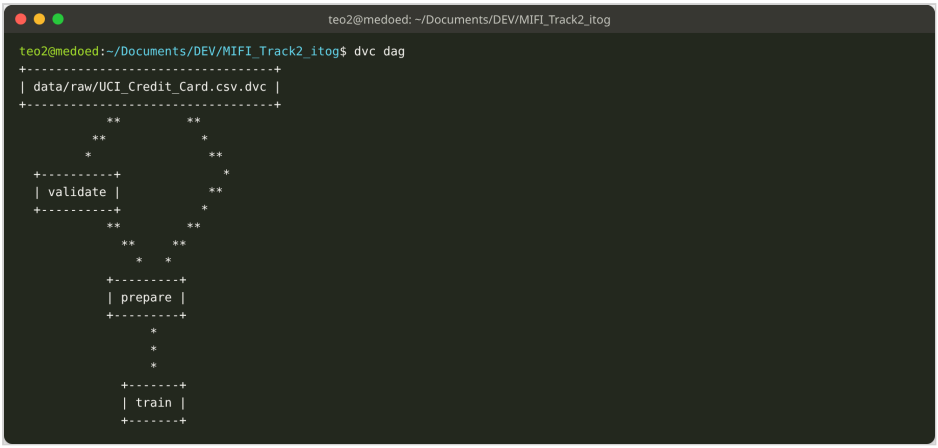


Рис. 5.1. Граф стадий

> `prepare` зависит от отчёта валидации, поэтому запускается только после успешной проверки данных.

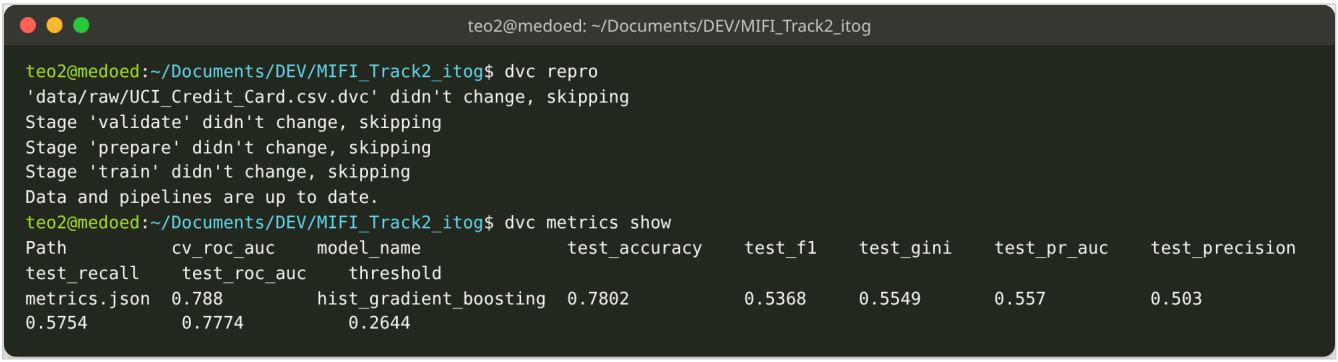


Рис. 5.2. Повторный dvc repro и метрики

> Ничего не изменилось – ничего не пересчитывается. Метрики DVC берёт из `metrics.json`.

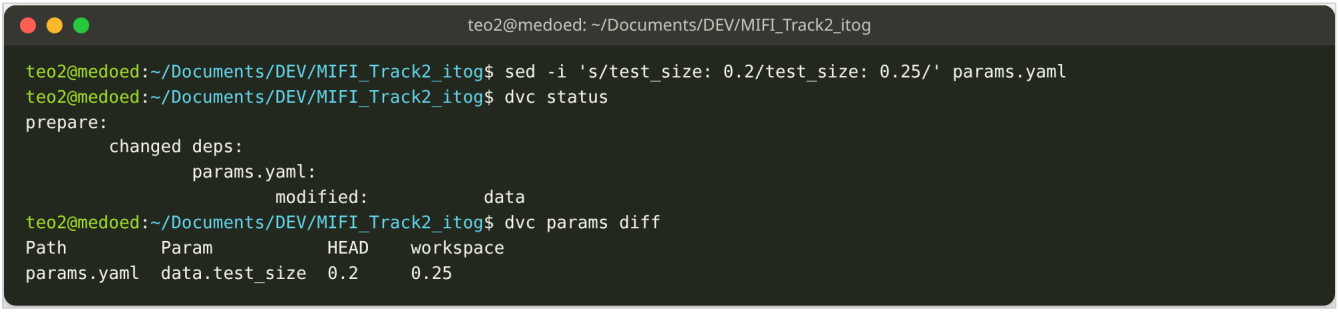


Рис. 5.3. Реакция на изменение параметра

> После правки `test_size` DVC видит, что изменился параметр стадии `prepare`, и пересчитает её и всё, что после. Правка потом откачена.

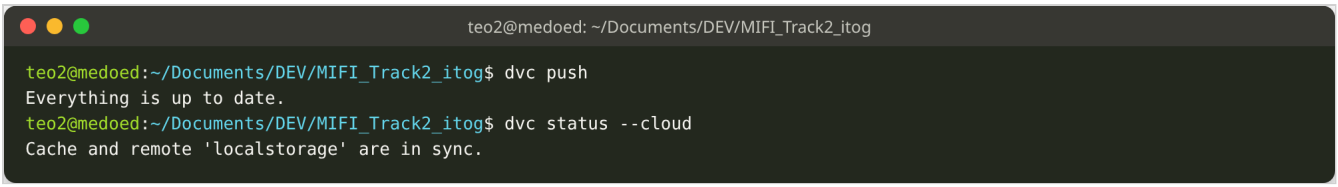


Рис. 5.4. Данные и модель синхронизированы с хранилищем DVC

> Хранилище DVC локальное, поэтому после клонирования с GitHub датасет берётся с Kaggle, а `dvc repro` собирает подготовленные данные и модель заново (инструкция в README). Проверил на свежем клоне без кэша DVC: пайплайн собрался с нуля за 3 минуты с теми же метриками.

6. Тестирование и CI

Unit-тесты на pytest покрывают очистку данных, признаки, валидацию, метрики, Pipeline, инференс, API и расчёт PSI. CI – GitHub Actions (`.github/workflows/ci-cd.yml`).



Рис. 6.1. Все тесты и линтеры локально

> 61 тест проходит, flake8 и black замечаний не дают.

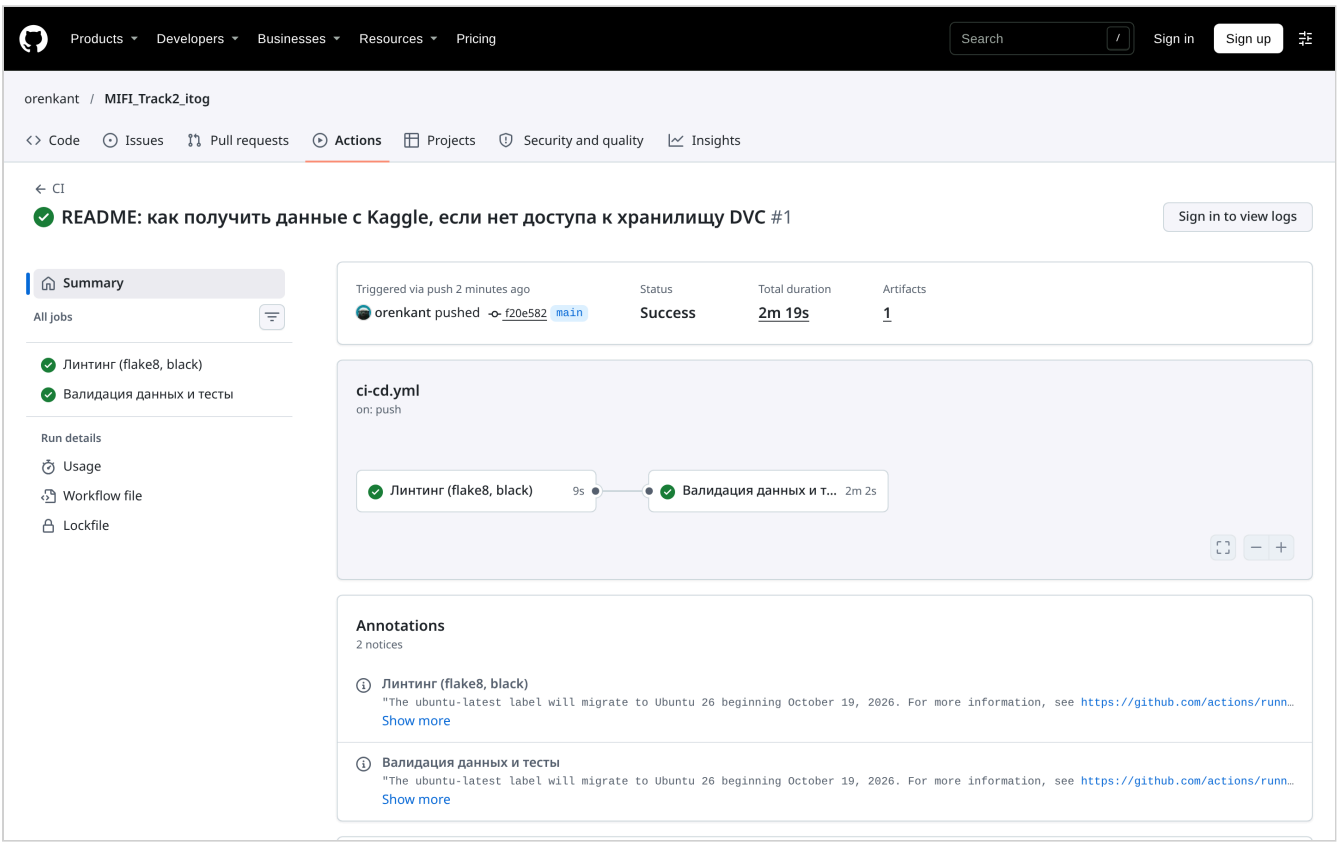


Рис. 6.2. Прогон GitHub Actions после push

> Сначала job с линтерами (flake8, black), после него – валидация данных и тесты. Оба зелёные с первого прогона, отчёт валидации сохранён как артефакт (Artifacts: 1). Уведомления в Annotations – про будущую смену образа ubuntu-latest, к проекту не относятся.

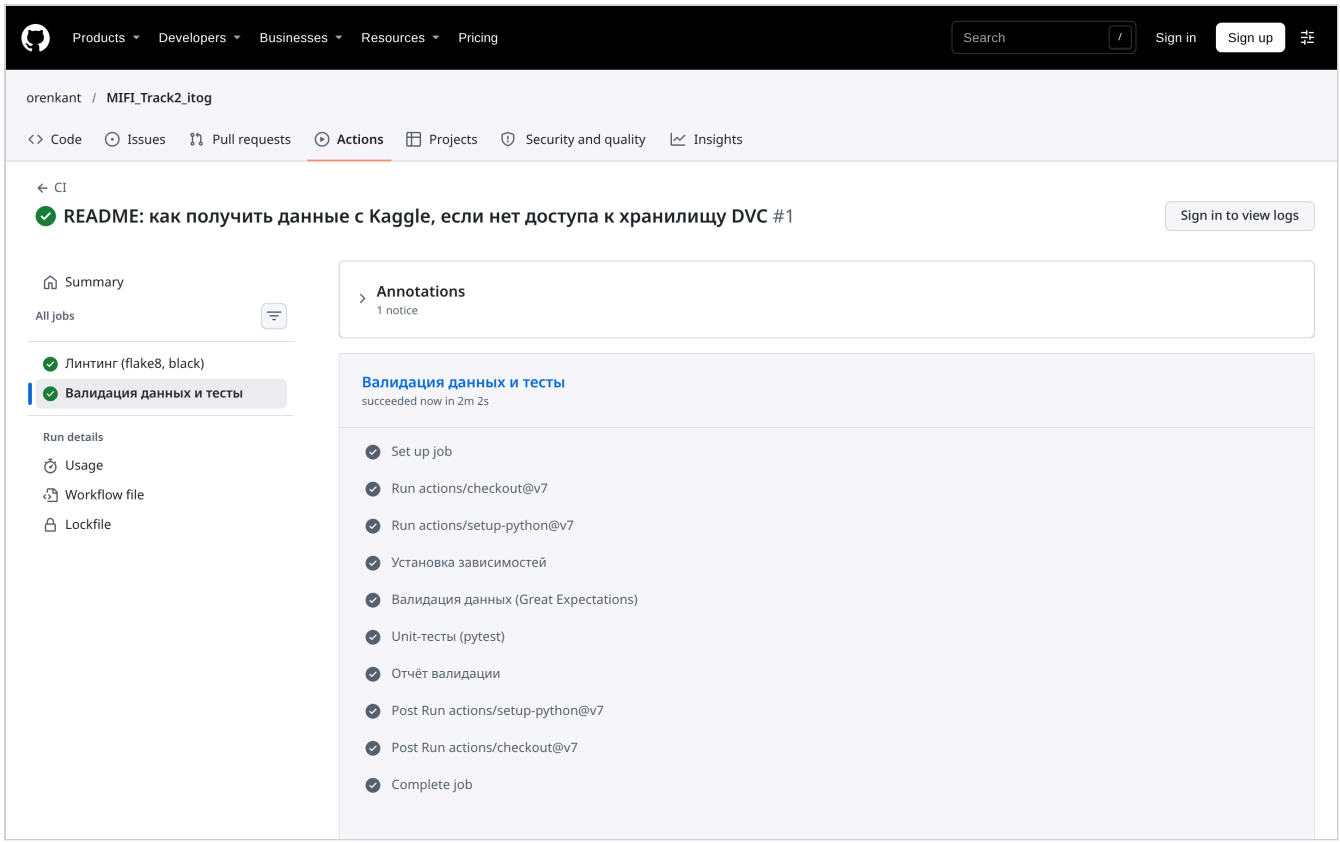


Рис. 6.3. Шаги job «Валидация данных и тесты»

> Без входа на GitHub логи шагов не показываются, поэтому ниже фрагмент лога этого job, скачанный через API.

GitHub Actions: лог job «Валидация данных и тесты» (фрагмент)

```
18:41:25  ▶ python -m src.data.validation tests/data/sample.csv --report reports/ci_validation_report.json
18:41:30  Валидация пройдена: tests/data/sample.csv, проверок 100
18:41:31  ▶ pytest -v
18:41:33  ============================= test session starts ==============================
18:41:33  platform linux -- Python 3.14.10, pytest-9.2.0, pluggy-1.7.0 --
/opt/hostedtoolcache/Python/3.14.10/x64/bin/python
18:41:33  cachedir: .pytest_cache
18:41:33  rootdir: /home/runner/work/MIFI_Track2_itog/MIFI_Track2_itog
18:41:33  configfile: pyproject.toml
18:41:33  testpaths: tests
18:41:33  plugins: anyio-4.16.0, platformdirs-4.14.5, hydra-core-1.3.8
18:41:37  collecting ... collected 61 items
18:41:37  tests/test_api.py::test_root PASSED                                    [  1%]
18:41:37  tests/test_api.py::test_health PASSED                                  [  3%]
18:41:37  tests/test_api.py::test_predict PASSED                                 [  4%]
          … ещё 50 тестов …
18:41:41  tests/test_validation.py::test_fails_on_unknown_category[SEX-3] PASSED   [ 88%]
18:41:41  tests/test_validation.py::test_fails_on_unknown_category[EDUCATION-9] PASSED [ 90%]
18:41:42  tests/test_validation.py::test_fails_on_unknown_category[PAY_0-12] PASSED [ 91%]
18:41:42  tests/test_validation.py::test_fails_on_bad_target PASSED               [ 93%]
18:41:43  tests/test_validation.py::test_fails_on_missing_column PASSED           [ 95%]
18:41:43  tests/test_validation.py::test_fails_on_wrong_type PASSED               [ 96%]
18:41:44  tests/test_validation.py::test_fails_on_duplicate_id PASSED             [ 98%]
18:41:45  tests/test_validation.py::test_new_data_without_target_and_id PASSED    [100%]
18:41:45  ============================== 61 passed in 11.62s ==============================
```

Рис. 6.4. Лог CI: валидация Great Expectations и pytest на раннере GitHub

> Полный датасет хранится в DVC, в CI проверяется выборка `tests/data/sample.csv` (1 000 строк исходных данных). На раннере (Python 3.14.10) прошли те же 61 тест, что и локально.

7. Docker и FastAPI

API (`src/api/app.py`): `/predict` для одного клиента, `/predict_batch` для списка, `/health` . Образ на `python:3.14-slim` , только зависимости для инференса.

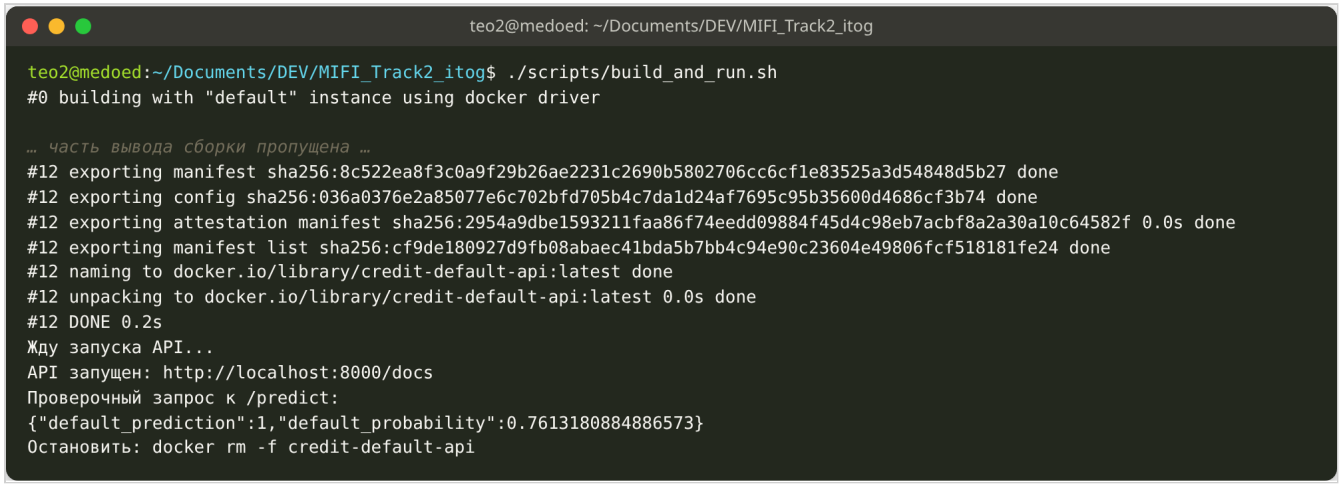


Рис. 7.1. Сборка образа и запуск контейнера скриптом

> Скрипт `scripts/build_and_run.sh` собирает образ, запускает контейнер, ждёт, пока API ответит на `/health` , и отправляет проверочный запрос.

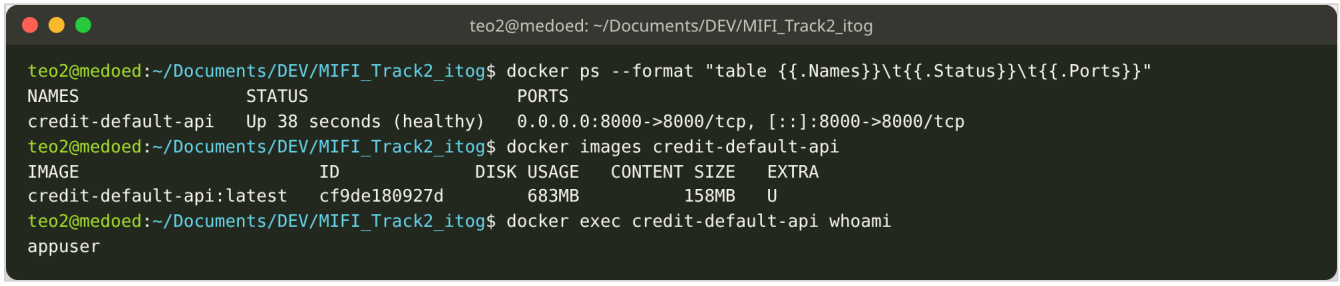


Рис. 7.2. Контейнер работает

> Статус `healthy` выставляет HEALTHCHECK из Dockerfile. Внутри процесс запущен не от root, а от пользователя `appuser` .

```
teo2@medoed: ~/Documents/DEV/MIFI_Track2_itog
teo2@medoed:~/Documents/DEV/MIFI_Track2_itog$ curl -s localhost:8000/health | python -m json.tool
{
    "status": "ok",
    "model_name": "hist_gradient_boosting",
    "threshold": 0.2643791761775945
}
teo2@medoed:~/Documents/DEV/MIFI_Track2_itog$ curl -s -X POST localhost:8000/predict -H "Content-Type: application/json" -d
@scripts/sample_request.json | python -m json.tool
{
    "default_prediction": 1,
    "default_probability": 0.7613180884886573
}
teo2@medoed:~/Documents/DEV/MIFI_Track2_itog$ sed 's/"AGE": 24/"AGE": 10/' scripts/sample_request.json | curl -s -X POST
localhost:8000/predict -H "Content-Type: application/json" -d @- -o /tmp/resp.json -w "HTTP %{http_code}\n"; python -m
json.tool /tmp/resp.json
HTTP 422
{
    "detail": [
        {
            "type": "greater_than_equal",
            "loc": [
                "body",
                "AGE"
            ],
            "msg": "Input should be greater than or equal to 18",
            "input": 10,
            "ctx": {
                "ge": 18
            }
        }
    ]
}
```

Рис. 7.3. Запросы к API

> Класс считается по порогу из обучения (0.264), а не по 0.5. На некорректный возраст API отвечает 422 с описанием ошибки: ограничения в схеме Pydantic те же, что в правилах Great Expectations.

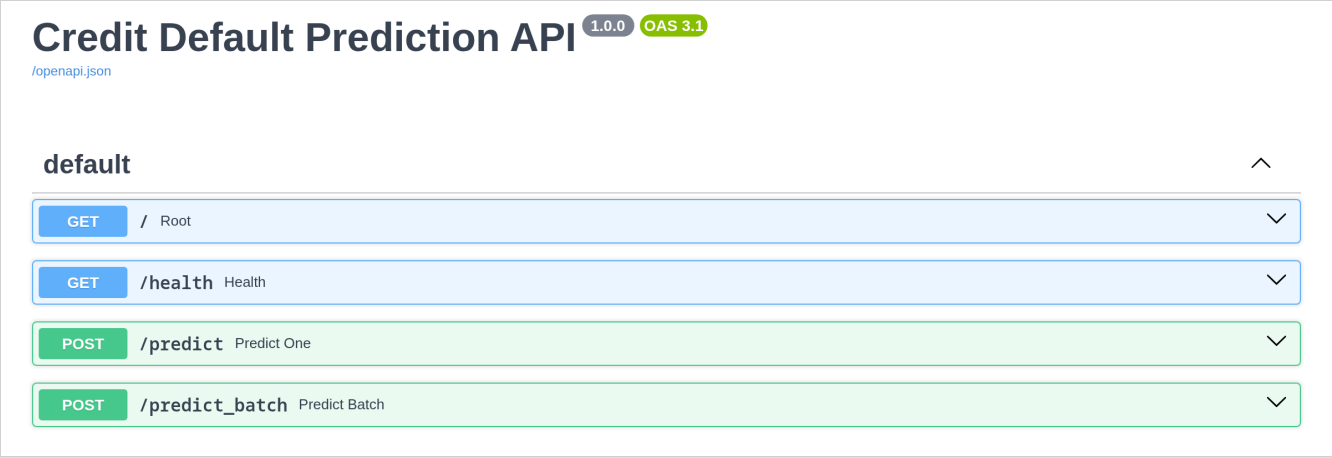


Рис. 7.4. Swagger UI (/docs): эндпоинты API

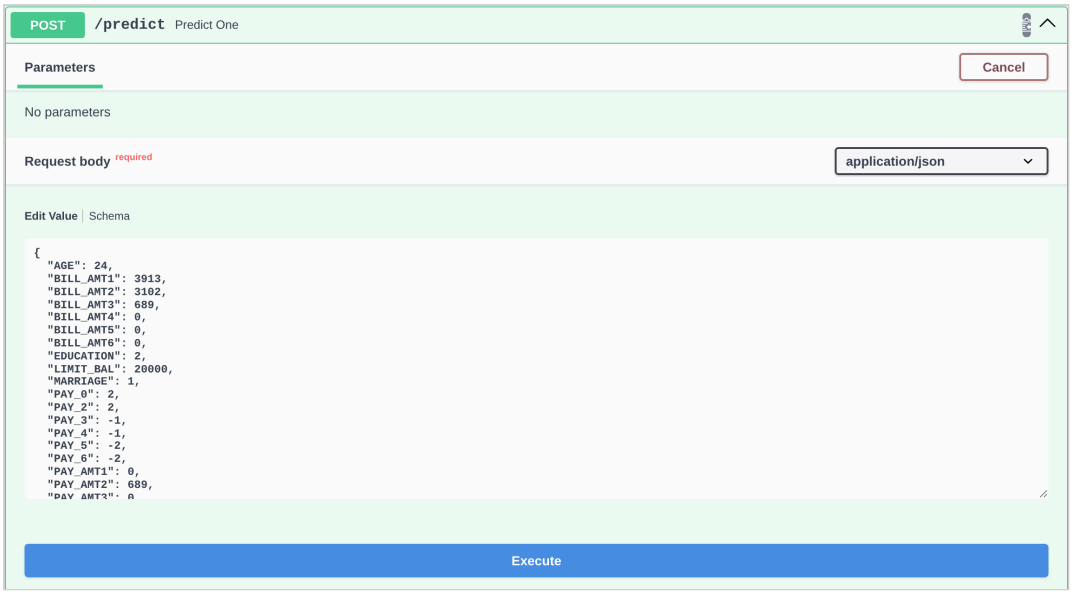


Рис. 7.5. Запрос к /predict из браузера

> Пример запроса подставляется из схемы. Это первый клиент датасета, у него в данных действительно дефолт.

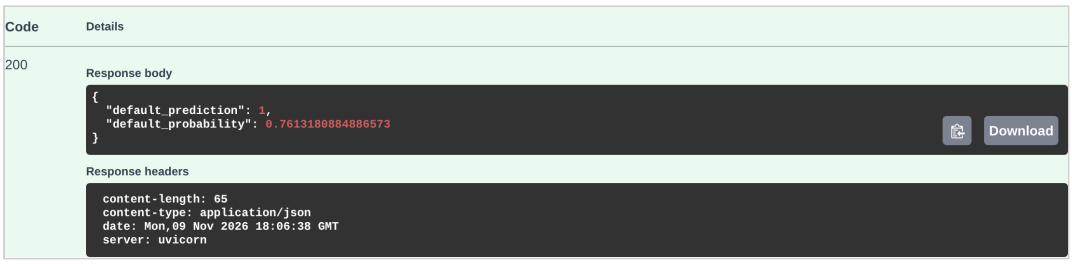


Рис. 7.6. Ответ API

> Вероятность дефолта 0.76 выше порога, поэтому класс 1.

8. Мониторинг дрифта

`src/monitoring/monitor.py` берёт 1 000 клиентов из тестовой выборки как «новый поток», проверяет их через Great Expectations, отправляет в API и считает PSI относительно train для ключевых признаков и для предсказанных вероятностей. Пороги: < 0.1 – stable, 0.1–0.25 – moderate, ≥ 0.25 – significant.

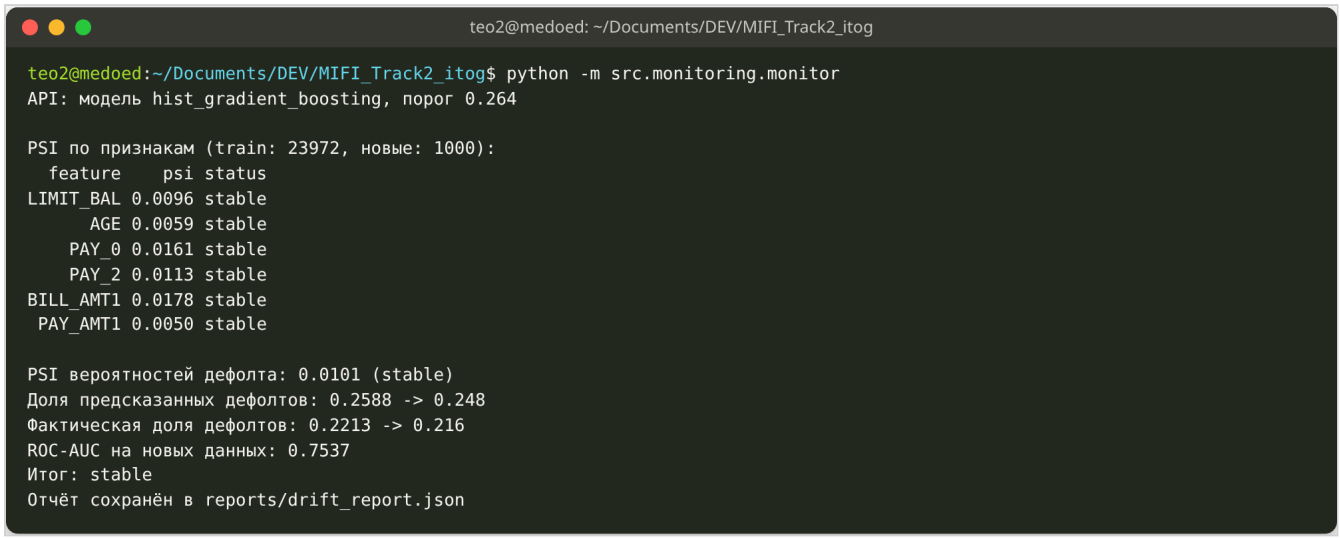


Рис. 8.1. Обычный поток

> Распределения совпадают с train, все PSI меньше 0.02, доля предсказанных дефолтов почти не изменилась.

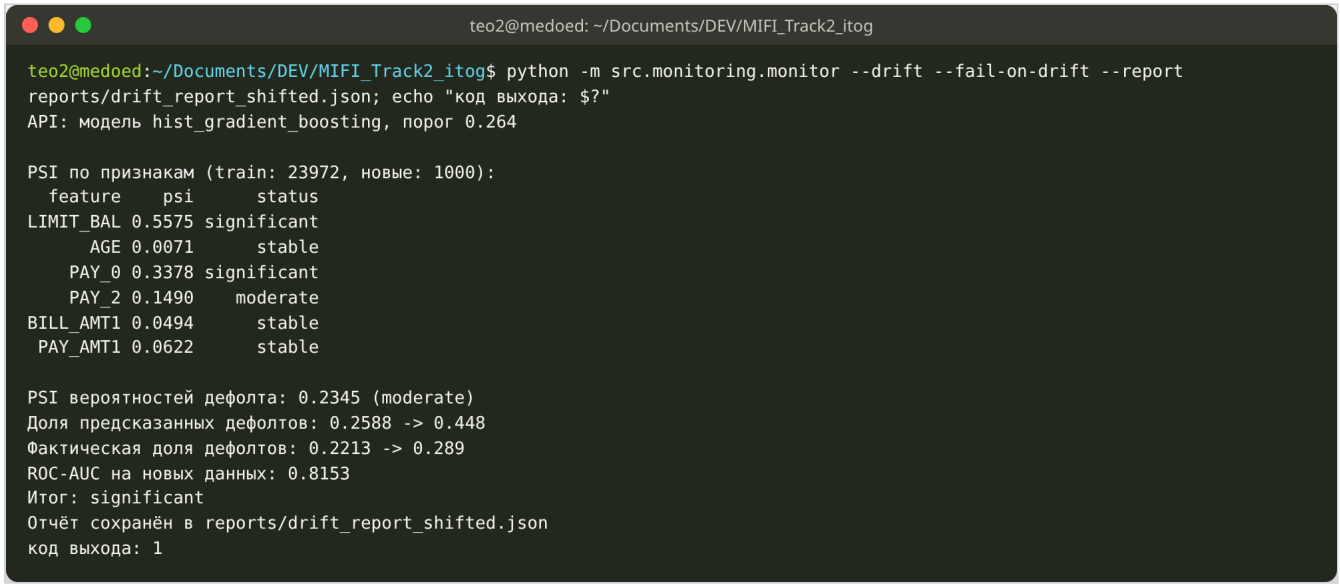


Рис. 8.2. Поток со сдвигом

> Сдвиг имитируется так: половина клиентов с просрочкой (в train около 23%) и лимиты снижены на 30%. Скрипт ловит сдвиг по `LIMIT_BAL` и `PAY_0`, PSI вероятностей вырос до 0.23, а с флагом `--fail-on-drift` возвращается код 1 – на это можно повесить переобучение.